Laboppgave 2: Objektorientert implementasjon av grafer

Sergiu Sturza · USN Kongsberg · Kildekode: github.com/Sergiu-Sturza/graf-prosjekt

Språk: C++17 · **Kompilator:** g++ · **Struktur:** `include/` (headere), `src/` (implementasjon), `tests/` (testsett), `docs/` (dokumentasjon)

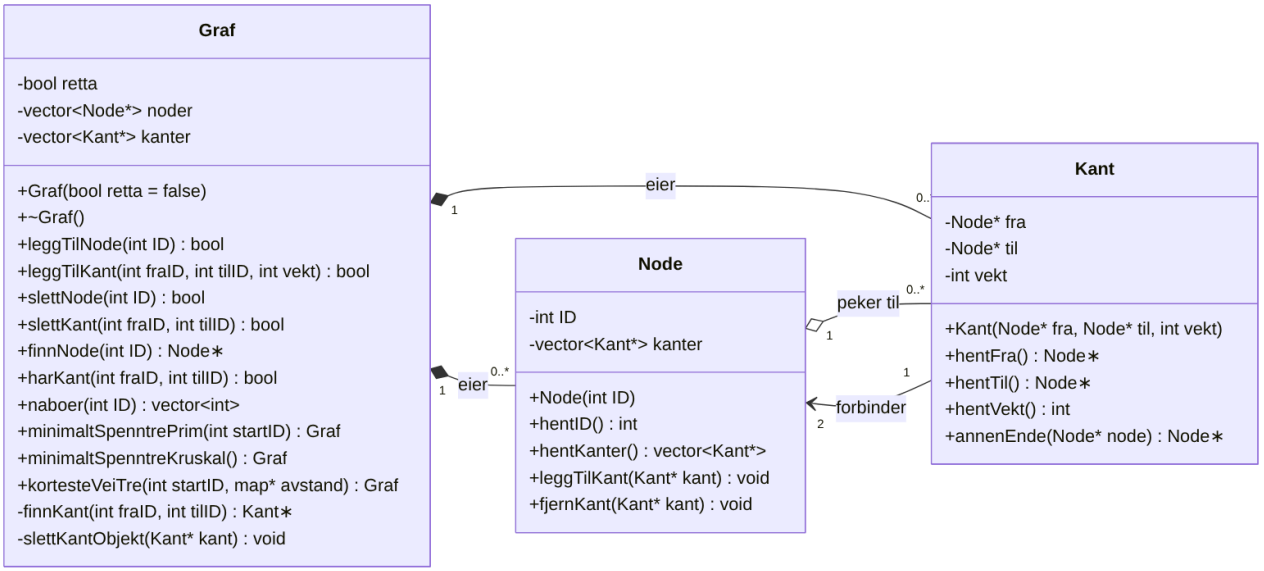
Oversikt og design

Grafen er implementert med tre klasser: `Graf`, `Node` og `Kant`. UML-klassediagram og designvalg ligger i UML-klassediagrammet under.

- `Graf` eier alle `Node`- og `Kant`-objektene (`std::vector<Node*>` og `std::vector<Kant*>`) og sletter dem i destruktøren.
- `Node` har en ID og en liste med pekere til `Kant`-objekter. Den eier ikke kantene.
- `Kant` peker på de to nodene den forbinder og har en vekt.
- Grafen er enten retta eller uretta (`Graf(bool retta = false)`).

Siden `Graf` eier rå pekere, er kopiering slått av (`= delete`) og flytting implementert. Det gjør at algoritmene kan returnere et nytt `Graf`-objekt effektivt.

UML-klassediagram



Figur 1: Klassediagram for Graf, Node og Kant. Graf eier alle Node- og Kant-objektene.

Del 1: Grunnleggende grafimplementasjon

1. Graf og Node. `Node` har `int ID` og en liste med naboer. `Graf` har en `vector` med nodepekere. I første versjon var nabolisten `vector<Node*>`. I Del 2 ble den erstattet av `vector<Kant*>`.

2. Legge til.

- `bool Graf::leggTilNode(int ID)` oppretter noden med `new Node(ID)`. Duplikat-ID avvises (returnerer `false`).
- `bool Graf::leggTilKant(int fraID, int tilID, int vekt = 1)` finner begge nodene med `finnNode`. Avvises hvis en node mangler, hvis kanten finnes fra før, eller hvis den er en løkke (`fra == til`).

3. Slette.

- `bool Graf::slettKant(int fraID, int tilID)` fjerner kanten fra nodenes lister og fra grafens kantliste, og kaller `delete`.
- `bool Graf::slettNode(int ID)` fjerner først alle kanter som er knyttet til noden, og sletter så selve noden. Ikke-eksisterende node eller kant gir `false`, ikke feil.

Del 2: Utvidet grafimplementasjon med kanter

1. Kant. `Kant` har pekere `fra` og `til`, `int vekt`, og `annenEnde(node)` som gir endepunktet som ikke er `node`.

2. Oppdatert Node og Graf. `Node` har nå `vector<Kant*> kanter` i stedet for direkte nabopekere. `leggTilKant` og `slettKant` jobber med `Kant`-objekter. Naboer finnes via `Graf::naboer(ID)`.

3. Retta og uretta grafer.

	Uretta	Retta
`leggTilKant(a, b)`	Én `Kant`, pekeren legges hos både a og b	Pekeren legges bare hos a
`slettKant(a, b)`	Finner kanten uansett retning	Finner bare a→b
`slettNode`	Fjerner alle kanter som berører noden	Fjerner både inngående og utgående kanter. Inngående kanter ligger ikke hos noden, så grafens kantliste brukes til å finne dem
Dijkstra	Følger alle kanter	Følger bare utgående kanter
Minimalt spenntre	Prim og Kruskal	Støttes ikke (`std::invalid_argument`)

Del 3: Ekstraoppgaver

1. Minimalt spenntre. Både Prim (`minimaltSpenntrePrim(int startID)`, med prioritetskø) og Kruskal (`minimaltSpenntreKruskal()`, sortering og union-find) er implementert. Begge returnerer et nytt `Graf`-objekt og endrer ikke originalen. For en usammenhengende graf gir Prim spenntreet for startnodens komponent, og Kruskal en spenntreskog.

2. Dijkstra. `kortesteVeiTre(int startID, std::map<int,int>* avstand = nullptr)` bruker en prioritetskø og returnerer korteste vei-treet som nytt `Graf`-objekt. Avstandene kan hentes ut via `avstand`. Negative vekter avvises med `std::invalid_argument`. Noder som ikke kan nås fra startnoden er ikke med i treet.

3. Retta grafer. Dijkstra fungerer uten endringer i algoritmen. Det eneste som må til, er at hver node bare har utgående kanter i `kanter`-listen, og at resultatgrafen settes til retta. Minimalt spenntre er definert for uretta grafer. For retta grafer trengs en annen algoritme (Chu-Liu/Edmonds), og funksjonene kaster derfor `std::invalid_argument`.

Testing

Testsettet fra oppgaven er implementert i `tests/testsett.cpp` (noder A–E har ID 1–5). Hele utskriften ligger i vedlegg A. Testene ble kompilert med `-Wall -Wextra -Wpedantic` og kjørt med AddressSanitizer og UBSan uten meldinger (ingen minnelekkasjer).

Test	Scenario	Resultat
1.1	Legge til fem noder	OK, fem noder med ID 1–5
1.2	Fjerne node 3	OK, fire noder, ID 3 er borte
2.1	Kantene 1-2, 1-3, 2-4, 3-5	OK, naboene stemmer for node 1, 2 og 3
2.2	Fjerne kanten 1-3	OK, node 1 har bare nabo 2, node 3 bare nabo 5
3.1	Prim fra A	OK, kantene A-B(1), B-C(1), D-E(2), B-D(4), total vekt 8
3.2	Kruskal	OK, samme kanter og vekt 8
4.1	Dijkstra fra A	OK, kantene A-C(2), C-E(3), E-D(4), A-B(4). Avstander B=4, C=2, D=9, E=5
5.1	Duplikat og sletting av ikke-eksisterende	OK, duplikat avvises, ingen feil
5.2	Slette alle noder og kanter	OK, grafen er tom
R.1–R.9	Egne tester for retta grafer, feilhåndtering og uavhengighet mellom original og resultatgraf	OK

Totalt: 44 OK, 0 feil.

Kompilering og kjøring

```
g++ -std=c++17 -Iinclude src/Graf.cpp src/Node.cpp main.cpp -o graf
.\graf.exe
g++ -std=c++17 -Iinclude src/Graf.cpp src/Node.cpp tests/testsett.cpp -o testsett
.\testsett.exe
```

Vedlegg A: Testutskrift

```
Testresultater - Laboppgave 2 (grafer)
Kompilert med: g++ -std=c++17 -Wall -Wextra -Wpedantic -fsanitize=address,undefined -Iinclude
src/*.cpp tests/testsett.cpp
AddressSanitizer og UBSan ga ingen meldinger (ingen minnelekkasjer eller udefinert
oppførsel).
Noder A-E i testsettet har ID 1-5.

Test 1: Legge til og fjerne noder
  [OK]   1.1 grafen har fem noder
  [OK]   1.1 nodene har ID 1, 2, 3, 4, 5
  [OK]   1.2 grafen har fire noder etter sletting
  [OK]   1.2 node 3 er ikke til stede
  [OK]   1.2 gjenværende noder er 1, 2, 4, 5

Test 2: Legge til og fjerne kanter
  [OK]   2.1 node 1 har naboene 2 og 3
  [OK]   2.1 node 2 har naboene 1 og 4
  [OK]   2.1 node 3 har naboene 1 og 5
  [OK]   2.1 grafen har fire kanter
  [OK]   2.2 node 1 har nå bare naboen 2
  [OK]   2.2 node 3 har nå bare naboen 5
  [OK]   2.2 grafen har tre kanter

Test 3: Minimalt spenntre
  [OK]   3.1 Prim: kantene er A-B(1), B-C(1), D-E(2), B-D(4)
  [OK]   3.1 Prim: total vekt er 8
  [OK]   3.1 Prim: 5 noder og 4 kanter
  [OK]   3.1 Prim fra C gir samme spenntre
  [OK]   3.2 Kruskal: samme kanter som Prim
  [OK]   3.2 Kruskal: total vekt er 8
  [OK]   3.x originalgrafen er uendret (5 noder, 7 kanter, vekt 22)

Test 4: Korteste vei (Dijkstra)
  [OK]   4.1 treet har kantene A-C(2), C-E(3), E-D(4), A-B(4)
  [OK]   4.1 avstand til B er 4
  [OK]   4.1 avstand til C er 2
  [OK]   4.1 avstand til D er 9 (via C og E)
  [OK]   4.1 avstand til E er 5 (via C)
  [OK]   4.x originalgrafen er uendret

Test 5: Robusthet
  [OK]   5.1 andre forsøk på å legge til ID 6 avvises
  [OK]   5.1 grafen har bare én node med ID 6
  [OK]   5.1 sletting av ikke-eksisterende node gir ingen feil
  [OK]   5.1 grafen er tom
  [OK]   5.1 sletting av ikke-eksisterende kant gir ingen feil
  [OK]   5.2 alle kanter slettet, nodene er igjen
  [OK]   5.2 alle noder slettet, grafen er tom
  [OK]   5.2 sletting av noder fjerner også kantene (tom graf, 0 kanter)

Ekstra: retta grafer (Del 2 og 3c)
  [OK]   R.1 kantene går bare én vei
  [OK]   R.2 kanten 1->2 finnes, 2->1 finnes ikke
  [OK]   R.3 sletting av kant i feil retning avvises
  [OK]   R.4 motsatt retning kan legges til som egen kant
  [OK]   R.5 sletting av node fjerner både inngående og utgående kanter
  [OK]   R.6 Dijkstra-treet er også retta
  [OK]   R.6 Dijkstra følger kantretningen (1->2->3 er kortere enn 1->3)
  [OK]   R.6 node 4 kan ikke nås fra 1 og er ikke med
  [OK]   R.7 minimalt spenntre på retta graf avvises med unntak
  [OK]   R.8 negative vekter avvises i Dijkstra
  [OK]   R.9 resultatgrafen er uavhengig av originalen
```

```
Resultat: 44 OK, 0 feil
```

Vedlegg B: Kildekode

include/Kant.h

```
#pragma once

class Node;

// En kant mellom to noder, med vekt.
// I en retta graf går kanten fra 'fra' til 'til'.
// I en uretta graf er rekkefølgen uten betydning.
class Kant {
private:
    Node* fra;
    Node* til;
    int vekt;

public:
    Kant(Node* fra, Node* til, int vekt = 1) : fra(fra), til(til), vekt(vekt) {}

    Node* hentFra() const { return fra; }
    Node* hentTil() const { return til; }
    int hentVekt() const { return vekt; }

    // Returnerer endepunktet som ikke er 'node'
    Node* annenEnde(const Node* node) const { return node == fra ? til : fra; }
};
```

include/Node.h

```
#pragma once
#include <vector>

class Kant;

// En node i grafen. Noden eier ikke kantene sine, det gjør Graf.
class Node {
private:
    int ID;
    std::vector<Kant*> kanter;   // kanter som går ut fra (eller, i uretta graf, inn til)
noden

public:
    explicit Node(int ID) : ID(ID) {}

    int hentID() const { return ID; }
    const std::vector<Kant*>& hentKanter() const { return kanter; }

    void leggTilKant(Kant* kant);
    void fjernKant(Kant* kant);
};
```

include/Graf.h

```
#pragma once
#include <cstddef>
#include <map>
#include <ostream>
#include <vector>
#include "Kant.h"
#include "Node.h"

// Graf eier alle Node- og Kant-objektene og sletter dem i destruktøren.
// Grafen er enten retta eller uretta (bestemmes når den lages).
```

```
class Graf {
private:
    bool retta;
    std::vector<Node*> noder;
    std::vector<Kant*> kanter;

    Kant* finnKant(int fraID, int tilID) const;
    void slettKantObjekt(Kant* kant);
    void ryddOpp();

public:
    explicit Graf(bool retta = false);
    ~Graf();

    // Grafen eier rå pekere, så den kan flyttes men ikke kopieres
    Graf(const Graf&) = delete;
    Graf& operator=(const Graf&) = delete;
    Graf(Graf&& other) noexcept;
    Graf& operator=(Graf&& other) noexcept;

    // Del 1 og 2: legge til og slette. Returnerer true hvis grafen ble endret.
    bool leggTilNode(int ID);
    bool leggTilKant(int fraID, int tilID, int vekt = 1);
    bool slettNode(int ID);
    bool slettKant(int fraID, int tilID);

    // Oppslag
    bool erRetta() const;
    Node* finnNode(int ID) const;
    bool harKant(int fraID, int tilID) const;
    std::vector<int> naboer(int ID) const;   // sorterte ID-er man kan gå til fra noden
    std::size_t antallNoder() const;
    std::size_t antallKanter() const;
    bool erTom() const;
    int totalVekt() const;
    const std::vector<Node*>& hentNoder() const;
    const std::vector<Kant*>& hentKanter() const;
    void skrivUt(std::ostream& ut) const;

    // Del 3: alle returnerer et nytt Graf-objekt og endrer ikke originalen.
    // Minimalt spenntre finnes bare for uretta grafer (kaster std::invalid_argument ellers).
    Graf minimaltSpenntrePrim(int startID) const;
    Graf minimaltSpenntreKruskal() const;
    // Fungerer for både retta og uretta grafer. Negative vekter kaster
std::invalid_argument.
    // Hvis 'avstand' er gitt, fylles den med korteste avstand fra startnoden til hver nåbar
node.
    Graf kortesteVeiTre(int startID, std::map<int, int>* avstand = nullptr) const;
};
```

src/Node.cpp

```
#include "Node.h"
#include <algorithm>

void Node::leggTilKant(Kant* kant) {
    kanter.push_back(kant);
}

void Node::fjernKant(Kant* kant) {
    kanter.erase(std::remove(kanter.begin(), kanter.end(), kant), kanter.end());
}
```

src/Graf.cpp

```
#include "Graf.h"
#include <algorithm>
#include <functional>
```

```cpp
#include <queue>
#include <set>
#include <stdexcept>
#include <utility>

namespace {

// Hjelpeklasse for Kruskal: holder styr på hvilke noder som allerede henger sammen
class UnionFind {
private:
    std::map<int, int> forelder;

public:
    void lagMengde(int x) { forelder[x] = x; }

    int finn(int x) {
        while (forelder[x] != x) {
            forelder[x] = forelder[forelder[x]];
            x = forelder[x];
        }
        return x;
    }

    // Returnerer false hvis a og b allerede er i samme mengde (kanten ville laget en syklus)
    bool slaaSammen(int a, int b) {
        int rotA = finn(a);
        int rotB = finn(b);
        if (rotA == rotB) {
            return false;
        }
        forelder[rotA] = rotB;
        return true;
    }
};

}  // namespace

// ---------- Konstruktører og destruktør ----------

Graf::Graf(bool retta) : retta(retta) {}

Graf::~Graf() {
    ryddOpp();
}

Graf::Graf(Graf&& other) noexcept
    : retta(other.retta), noder(std::move(other.noder)), kanter(std::move(other.kanter)) {
    other.noder.clear();
    other.kanter.clear();
}

Graf& Graf::operator=(Graf&& other) noexcept {
    if (this != &other) {
        ryddOpp();
        retta = other.retta;
        noder = std::move(other.noder);
        kanter = std::move(other.kanter);
        other.noder.clear();
        other.kanter.clear();
    }
    return *this;
}

void Graf::ryddOpp() {
    for (Kant* kant : kanter) {
        delete kant;
    }
    for (Node* node : noder) {
        delete node;
    }
```

```
    kanter.clear();
    noder.clear();
}

// ---------- Legge til ----------

bool Graf::leggTilNode(int ID) {
    if (finnNode(ID) != nullptr) {
        return false;   // ingen duplikater
    }
    noder.push_back(new Node(ID));
    return true;
}

bool Graf::leggTilKant(int fraID, int tilID, int vekt) {
    if (fraID == tilID) {
        return false;   // ingen løkker
    }
    Node* fra = finnNode(fraID);
    Node* til = finnNode(tilID);
    if (fra == nullptr || til == nullptr || finnKant(fraID, tilID) != nullptr) {
        return false;
    }

    Kant* kant = new Kant(fra, til, vekt);
    kanter.push_back(kant);
    fra->leggTilKant(kant);
    if (!retta) {
        til->leggTilKant(kant);   // uretta: samme kant nås fra begge ender
    }
    return true;
}

// ---------- Slette ----------

void Graf::slettKantObjekt(Kant* kant) {
    kant->hentFra()->fjernKant(kant);
    if (!retta) {
        kant->hentTil()->fjernKant(kant);
    }
    kanter.erase(std::remove(kanter.begin(), kanter.end(), kant), kanter.end());
    delete kant;
}

bool Graf::slettKant(int fraID, int tilID) {
    Kant* kant = finnKant(fraID, tilID);
    if (kant == nullptr) {
        return false;
    }
    slettKantObjekt(kant);
    return true;
}

bool Graf::slettNode(int ID) {
    Node* node = finnNode(ID);
    if (node == nullptr) {
        return false;
    }

    // Fjern alle kanter som er knyttet til noden (både inn og ut)
    std::vector<Kant*> kopi = kanter;
    for (Kant* kant : kopi) {
        if (kant->hentFra() == node || kant->hentTil() == node) {
            slettKantObjekt(kant);
        }
    }

    noder.erase(std::remove(noder.begin(), noder.end(), node), noder.end());
    delete node;
    return true;
```

```
}

// ---------- Oppslag ----------

Node* Graf::finnNode(int ID) const {
    for (Node* node : noder) {
        if (node->hentID() == ID) {
            return node;
        }
    }
    return nullptr;
}

Kant* Graf::finnKant(int fraID, int tilID) const {
    for (Kant* kant : kanter) {
        int a = kant->hentFra()->hentID();
        int b = kant->hentTil()->hentID();
        if (a == fraID && b == tilID) {
            return kant;
        }
        if (!retta && a == tilID && b == fraID) {
            return kant;
        }
    }
    return nullptr;
}

bool Graf::harKant(int fraID, int tilID) const {
    return finnKant(fraID, tilID) != nullptr;
}

std::vector<int> Graf::naboer(int ID) const {
    std::vector<int> resultat;
    Node* node = finnNode(ID);
    if (node == nullptr) {
        return resultat;
    }
    for (Kant* kant : node->hentKanter()) {
        resultat.push_back(kant->annenEnde(node)->hentID());
    }
    std::sort(resultat.begin(), resultat.end());
    return resultat;
}

bool Graf::erRetta() const { return retta; }
std::size_t Graf::antallNoder() const { return noder.size(); }
std::size_t Graf::antallKanter() const { return kanter.size(); }
bool Graf::erTom() const { return noder.empty() && kanter.empty(); }
const std::vector<Node*>& Graf::hentNoder() const { return noder; }
const std::vector<Kant*>& Graf::hentKanter() const { return kanter; }

int Graf::totalVekt() const {
    int sum = 0;
    for (Kant* kant : kanter) {
        sum += kant->hentVekt();
    }
    return sum;
}

void Graf::skrivUt(std::ostream& ut) const {
    for (Node* node : noder) {
        ut << node->hentID() << ":";
        for (Kant* kant : node->hentKanter()) {
            ut << ' ' << kant->annenEnde(node)->hentID() << '(' << kant->hentVekt() << ')';
        }
        ut << '\n';
    }
}

// ---------- Del 3: algoritmer ----------
```

```
Graf Graf::minimaltSpenntrePrim(int startID) const {
    if (retta) {
        throw std::invalid_argument("Minimalt spenntre er kun definert for uretta grafer");
    }

    Graf mst(false);
    Node* start = finnNode(startID);
    if (start == nullptr) {
        return mst;
    }

    struct Kandidat {
        int vekt;
        Kant* kant;
        Node* til;   // noden utenfor treet som kanten fører til
    };
    auto storre = [](const Kandidat& a, const Kandidat& b) { return a.vekt > b.vekt; };
    std::priority_queue<Kandidat, std::vector<Kandidat>, decltype(storre)> ko(storre);

    std::set<int> iTreet;
    iTreet.insert(startID);
    mst.leggTilNode(startID);
    for (Kant* kant : start->hentKanter()) {
        ko.push({kant->hentVekt(), kant, kant->annenEnde(start)});
    }

    while (!ko.empty()) {
        Kandidat valgt = ko.top();
        ko.pop();

        int tilID = valgt.til->hentID();
        if (iTreet.count(tilID) > 0) {
            continue;   // noden er allerede med
        }

        iTreet.insert(tilID);
        mst.leggTilNode(tilID);
        mst.leggTilKant(valgt.kant->hentFra()->hentID(), valgt.kant->hentTil()->hentID(),
                        valgt.kant->hentVekt());

        for (Kant* kant : valgt.til->hentKanter()) {
            Node* annen = kant->annenEnde(valgt.til);
            if (iTreet.count(annen->hentID()) == 0) {
                ko.push({kant->hentVekt(), kant, annen});
            }
        }
    }
    return mst;
}

Graf Graf::minimaltSpenntreKruskal() const {
    if (retta) {
        throw std::invalid_argument("Minimalt spenntre er kun definert for uretta grafer");
    }

    Graf mst(false);
    UnionFind grupper;
    for (Node* node : noder) {
        mst.leggTilNode(node->hentID());
        grupper.lagMengde(node->hentID());
    }

    std::vector<Kant*> sortert = kanter;
    std::stable_sort(sortert.begin(), sortert.end(),
                     [](const Kant* a, const Kant* b) { return a->hentVekt() < b->hentVekt();
});

    for (Kant* kant : sortert) {
        int a = kant->hentFra()->hentID();
```

```
        int b = kant->hentTil()->hentID();
        if (grupper.slaaSammen(a, b)) {
            mst.leggTilKant(a, b, kant->hentVekt());
        }
    }
    return mst;
}

Graf Graf::kortesteVeiTre(int startID, std::map<int, int>* avstandUt) const {
    Graf tre(retta);
    Node* start = finnNode(startID);
    if (start == nullptr) {
        return tre;
    }
    for (Kant* kant : kanter) {
        if (kant->hentVekt() < 0) {
            throw std::invalid_argument("Dijkstra støtter ikke negative vekter");
        }
    }

    std::map<int, int> avstand;       // beste kjente avstand fra start
    std::map<int, Kant*> forelder;    // kanten som ga beste avstand
    std::set<int> ferdig;

    using Par = std::pair<int, int>;  // (avstand, node-ID)
    std::priority_queue<Par, std::vector<Par>, std::greater<Par>> ko;
    avstand[startID] = 0;
    ko.push({0, startID});

    while (!ko.empty()) {
        Par topp = ko.top();
        ko.pop();
        int d = topp.first;
        int id = topp.second;
        if (ferdig.count(id) > 0) {
            continue;
        }
        ferdig.insert(id);

        Node* u = finnNode(id);
        // I en retta graf ligger bare utgående kanter i hentKanter(), så retningen
respekteres
        for (Kant* kant : u->hentKanter()) {
            Node* v = kant->annenEnde(u);
            int nyAvstand = d + kant->hentVekt();
            auto it = avstand.find(v->hentID());
            if (it == avstand.end() || nyAvstand < it->second) {
                avstand[v->hentID()] = nyAvstand;
                forelder[v->hentID()] = kant;
                ko.push({nyAvstand, v->hentID()});
            }
        }
    }

    for (const auto& par : avstand) {
        tre.leggTilNode(par.first);
    }
    for (const auto& par : forelder) {
        Kant* kant = par.second;
        tre.leggTilKant(kant->hentFra()->hentID(), kant->hentTil()->hentID(), kant-
>hentVekt());
    }

    if (avstandUt != nullptr) {
        *avstandUt = avstand;
    }
    return tre;
}
```

main.cpp

```
#include <iostream>
#include <map>
#include "Graf.h"

int main() {
    // Eksempelgraf fra Test 3 (A-E = 1-5)
    Graf g;
    for (int id = 1; id <= 5; ++id) {
        g.leggTilNode(id);
    }
    g.leggTilKant(1, 2, 1);
    g.leggTilKant(1, 3, 3);
    g.leggTilKant(2, 3, 1);
    g.leggTilKant(2, 4, 4);
    g.leggTilKant(3, 4, 5);
    g.leggTilKant(3, 5, 6);
    g.leggTilKant(4, 5, 2);

    std::cout << "Original graf:\n";
    g.skrivUt(std::cout);

    Graf mst = g.minimaltSpenntrePrim(1);
    std::cout << "\nMinimalt spenntre (Prim fra node 1), total vekt " << mst.totalVekt() <<
":\n";
    mst.skrivUt(std::cout);

    std::map<int, int> avstand;
    Graf tre = g.kortesteVeiTre(1, &avstand);
    std::cout << "\nKorteste vei-tre (Dijkstra fra node 1):\n";
    tre.skrivUt(std::cout);
    for (const auto& par : avstand) {
        std::cout << "  avstand til " << par.first << ": " << par.second << '\n';
    }
    return 0;
}
```

tests/testsett.cpp

```
// Testsett for Laboppgave 2 (test 1-5 fra oppgaven + ekstra tester for retta grafer)
#include <algorithm>
#include <iostream>
#include <map>
#include <set>
#include <stdexcept>
#include <string>
#include <tuple>
#include <vector>
#include "Graf.h"

namespace {

int antallOK = 0;
int antallFeil = 0;

void overskrift(const std::string& tekst) {
    std::cout << '\n' << tekst << '\n';
}

void sjekk(bool betingelse, const std::string& beskrivelse) {
    if (betingelse) {
        ++antallOK;
        std::cout << "  [OK]   " << beskrivelse << '\n';
    } else {
        ++antallFeil;
        std::cout << "  [FEIL] " << beskrivelse << '\n';
    }
```

```
}

std::vector<int> nodeIDer(const Graf& g) {
    std::vector<int> ider;
    for (Node* node : g.hentNoder()) {
        ider.push_back(node->hentID());
    }
    std::sort(ider.begin(), ider.end());
    return ider;
}

using Kantesett = std::set<std::tuple<int, int, int>>;

// Uretta kanter normaliseres slik at (1,2) og (2,1) blir samme kant
Kantesett kantesett(const Graf& g) {
    Kantesett resultat;
    for (Kant* kant : g.hentKanter()) {
        int a = kant->hentFra()->hentID();
        int b = kant->hentTil()->hentID();
        if (!g.erRetta() && a > b) {
            std::swap(a, b);
        }
        resultat.insert(std::make_tuple(a, b, kant->hentVekt()));
    }
    return resultat;
}

// Noder A-E har ID 1-5
constexpr int A = 1, B = 2, C = 3, D = 4, E = 5;

Graf lagMstGraf() {
    Graf g;
    for (int id : {A, B, C, D, E}) g.leggTilNode(id);
    g.leggTilKant(A, B, 1);
    g.leggTilKant(A, C, 3);
    g.leggTilKant(B, C, 1);
    g.leggTilKant(B, D, 4);
    g.leggTilKant(C, D, 5);
    g.leggTilKant(C, E, 6);
    g.leggTilKant(D, E, 2);
    return g;
}

Graf lagDijkstraGraf(bool retta = false) {
    Graf g(retta);
    for (int id : {A, B, C, D, E}) g.leggTilNode(id);
    g.leggTilKant(A, B, 4);
    g.leggTilKant(A, C, 2);
    g.leggTilKant(B, C, 5);
    g.leggTilKant(B, D, 10);
    g.leggTilKant(C, E, 3);
    g.leggTilKant(D, E, 4);
    return g;
}

Graf lagGrafMedFemNoder() {
    Graf g;
    for (int id = 1; id <= 5; ++id) g.leggTilNode(id);
    return g;
}

using IDer = std::vector<int>;

void test1() {
    overskrift("Test 1: Legge til og fjerne noder");

    Graf g;
    for (int id = 1; id <= 5; ++id) g.leggTilNode(id);
    sjekk(g.antallNoder() == 5, "1.1 grafen har fem noder");
    sjekk(nodeIDer(g) == IDer({1, 2, 3, 4, 5}), "1.1 nodene har ID 1, 2, 3, 4, 5");
```

```
    g.slettNode(3);
    sjekk(g.antallNoder() == 4, "1.2 grafen har fire noder etter sletting");
    sjekk(g.finnNode(3) == nullptr, "1.2 node 3 er ikke til stede");
    sjekk(nodeIDer(g) == IDer({1, 2, 4, 5}), "1.2 gjenværende noder er 1, 2, 4, 5");
}

void test2() {
    overskrift("Test 2: Legge til og fjerne kanter");

    Graf g = lagGrafMedFemNoder();
    g.leggTilKant(1, 2);
    g.leggTilKant(1, 3);
    g.leggTilKant(2, 4);
    g.leggTilKant(3, 5);
    sjekk(g.naboer(1) == IDer({2, 3}), "2.1 node 1 har naboene 2 og 3");
    sjekk(g.naboer(2) == IDer({1, 4}), "2.1 node 2 har naboene 1 og 4");
    sjekk(g.naboer(3) == IDer({1, 5}), "2.1 node 3 har naboene 1 og 5");
    sjekk(g.antallKanter() == 4, "2.1 grafen har fire kanter");

    g.slettKant(1, 3);
    sjekk(g.naboer(1) == IDer({2}), "2.2 node 1 har nå bare naboen 2");
    sjekk(g.naboer(3) == IDer({5}), "2.2 node 3 har nå bare naboen 5");
    sjekk(g.antallKanter() == 3, "2.2 grafen har tre kanter");
}

void test3() {
    overskrift("Test 3: Minimalt spenntre");

    Kantesett forventet = {std::make_tuple(A, B, 1), std::make_tuple(B, C, 1),
                           std::make_tuple(D, E, 2), std::make_tuple(B, D, 4)};

    Graf g = lagMstGraf();
    Graf prim = g.minimaltSpenntrePrim(A);
    sjekk(kantesett(prim) == forventet, "3.1 Prim: kantene er A-B(1), B-C(1), D-E(2), B-
D(4)");
    sjekk(prim.totalVekt() == 8, "3.1 Prim: total vekt er 8");
    sjekk(prim.antallNoder() == 5 && prim.antallKanter() == 4, "3.1 Prim: 5 noder og 4
kanter");

    Graf prim2 = g.minimaltSpenntrePrim(C);
    sjekk(kantesett(prim2) == forventet, "3.1 Prim fra C gir samme spenntre");

    Graf kruskal = g.minimaltSpenntreKruskal();
    sjekk(kantesett(kruskal) == forventet, "3.2 Kruskal: samme kanter som Prim");
    sjekk(kruskal.totalVekt() == 8, "3.2 Kruskal: total vekt er 8");

    sjekk(g.antallNoder() == 5 && g.antallKanter() == 7 && g.totalVekt() == 22,
          "3.x originalgrafen er uendret (5 noder, 7 kanter, vekt 22)");
}

void test4() {
    overskrift("Test 4: Korteste vei (Dijkstra)");

    Kantesett forventet = {std::make_tuple(A, C, 2), std::make_tuple(C, E, 3),
                           std::make_tuple(D, E, 4), std::make_tuple(A, B, 4)};

    Graf g = lagDijkstraGraf();
    std::map<int, int> avstand;
    Graf tre = g.kortesteVeiTre(A, &avstand);

    sjekk(kantesett(tre) == forventet, "4.1 treet har kantene A-C(2), C-E(3), E-D(4), A-
B(4)");
    sjekk(avstand[B] == 4, "4.1 avstand til B er 4");
    sjekk(avstand[C] == 2, "4.1 avstand til C er 2");
    sjekk(avstand[D] == 9, "4.1 avstand til D er 9 (via C og E)");
    sjekk(avstand[E] == 5, "4.1 avstand til E er 5 (via C)");
    sjekk(g.antallKanter() == 6 && g.totalVekt() == 28, "4.x originalgrafen er uendret");
}
```

```
void test5() {
    overskrift("Test 5: Robusthet");

    Graf g;
    bool forste = g.leggTilNode(6);
    bool andre = g.leggTilNode(6);
    g.leggTilNode(6);
    sjekk(forste && !andre, "5.1 andre forsøk på å legge til ID 6 avvises");
    sjekk(g.antallNoder() == 1, "5.1 grafen har bare én node med ID 6");

    bool slettet1 = g.slettNode(6);
    bool slettet2 = g.slettNode(6);
    bool slettet3 = g.slettNode(6);
    sjekk(slettet1 && !slettet2 && !slettet3, "5.1 sletting av ikke-eksisterende node gir
ingen feil");
    sjekk(g.erTom(), "5.1 grafen er tom");
    sjekk(!g.slettKant(1, 2), "5.1 sletting av ikke-eksisterende kant gir ingen feil");

    Graf h = lagMstGraf();
    for (Kant* kant : std::vector<Kant*>(h.hentKanter())) {
        h.slettKant(kant->hentFra()->hentID(), kant->hentTil()->hentID());
    }
    sjekk(h.antallKanter() == 0 && h.antallNoder() == 5, "5.2 alle kanter slettet, nodene er
igjen");
    for (int id : {A, B, C, D, E}) h.slettNode(id);
    sjekk(h.erTom(), "5.2 alle noder slettet, grafen er tom");

    Graf k = lagMstGraf();
    for (int id : {A, B, C, D, E}) k.slettNode(id);
    sjekk(k.erTom(), "5.2 sletting av noder fjerner også kantene (tom graf, 0 kanter)");
}

void testRetta() {
    overskrift("Ekstra: retta grafer (Del 2 og 3c)");

    Graf g(true);
    for (int id = 1; id <= 3; ++id) g.leggTilNode(id);
    g.leggTilKant(1, 2, 1);
    g.leggTilKant(2, 3, 1);
    sjekk(g.naboer(1) == IDer({2}) && g.naboer(2) == IDer({3}) && g.naboer(3).empty(),
          "R.1 kantene går bare én vei");
    sjekk(g.harKant(1, 2) && !g.harKant(2, 1), "R.2 kanten 1->2 finnes, 2->1 finnes ikke");
    sjekk(!g.slettKant(2, 1), "R.3 sletting av kant i feil retning avvises");
    sjekk(g.leggTilKant(2, 1, 1) && g.antallKanter() == 3, "R.4 motsatt retning kan legges
til som egen kant");

    g.slettNode(2);
    sjekk(g.antallKanter() == 0 && g.naboer(1).empty(), "R.5 sletting av node fjerner både
inngående og utgående kanter");

    Graf d(true);
    for (int id = 1; id <= 4; ++id) d.leggTilNode(id);
    d.leggTilKant(1, 2, 1);
    d.leggTilKant(2, 3, 1);
    d.leggTilKant(1, 3, 5);
    d.leggTilKant(4, 1, 1);
    std::map<int, int> avstand;
    Graf tre = d.kortesteVeiTre(1, &avstand);
    sjekk(tre.erRetta(), "R.6 Dijkstra-treet er også retta");
    sjekk(kantesett(tre) == Kantesett({std::make_tuple(1, 2, 1), std::make_tuple(2, 3, 1)}),
          "R.6 Dijkstra følger kantretningen (1->2->3 er kortere enn 1->3)");
    sjekk(avstand[3] == 2 && avstand.count(4) == 0, "R.6 node 4 kan ikke nås fra 1 og er ikke
med");

    bool kastet = false;
    try {
        d.minimaltSpenntrePrim(1);
    } catch (const std::invalid_argument&) {
        kastet = true;
    }
```

```
    sjekk(kastet, "R.7 minimalt spenntre på retta graf avvises med unntak");

    Graf neg;
    neg.leggTilNode(1);
    neg.leggTilNode(2);
    neg.leggTilKant(1, 2, -1);
    kastet = false;
    try {
        neg.kortesteVeiTre(1);
    } catch (const std::invalid_argument&) {
        kastet = true;
    }
    sjekk(kastet, "R.8 negative vekter avvises i Dijkstra");

    Graf kilde = lagMstGraf();
    Graf kopi = kilde.minimaltSpenntrePrim(A);
    kilde.slettNode(A);
    sjekk(kopi.antallNoder() == 5 && kopi.antallKanter() == 4,
          "R.9 resultatgrafen er uavhengig av originalen");
}

}  // namespace

int main() {
    test1();
    test2();
    test3();
    test4();
    test5();
    testRetta();

    std::cout << "\nResultat: " << antallOK << " OK, " << antallFeil << " feil\n";
    return antallFeil == 0 ? 0 : 1;
}
```